AI Model to Predict the ICO (Cryptocurrencies) offerings for Fundraising teams and Startups Using Machine Learning Algorithms

(https://github.com/Thesineo/siner)

Table of Contents

1. Introduction **4**
2.Data Exploration **6**
3.Modelling Techniques **9**
3.1 K-nearest neighbours (KNN) 9
3.2 Naive Bayes (NB) 9
3.3 Logistic Regression (LR) 10
3.4 Support Vector Machines (SVM) 10
3.5 Decision Tree (DT) 10
3.6 Random Forest (RF) 11
4. Evaluation Metrics **12**
4.1 Precision 12
4.2 Recall 12
4.3 ROC 12
4.4 F-measure 13
4.5 Kappa 13
5. Effective comparison **15**
5.1 KNN vs Baseline Classifiers 20
5.2 KNN vs Highline Classifiers 20
6. Conclusion **21**
7. References **22**
8.Appendix **24**
Pre-Processing of Data 24
KNN Classifier 25
Naive Bayes Classifier 27
Decision Tree 28
Random Forest 29
Support Vector Mechanism 30
Logistic Regression 31

1. Introduction

Crypto platforms acceptance and market capitalization have seen a decent increase in the last few years . The crowdfunding of these currencies does not need to follow the standardised procedure of Venture Capital investment due to their decentralised nature. Formerly, they go through initial Coin Offerings (ICOs) (Chohan,2017). Freedom of this crowdfunding opens new doors for deceitful operations to function ICOs for “pump and dump” schemes. Even though ICOs offer fair and lawful investment opportunities, it is difficult to differentiate between the fraudulent and non-fraudulent projects.As its decentralised nature imposes significant hurdles to government regulation, making a prosperous atmosphere for investors money. Due to this high information asymmetries, investors find it difficult to assess the success rate of the companies raising funds through ICOs (Momtaz,2020).

In this context, a data-driven approach to investment decision-making can provide effective results (Ferrari and Muffatto,2021). As increased computational capacity ,the development of algorithms and large amount of available data have given a spark to machine learning (ML) applications (Agarwal,2020) ,development has been so efficient that few authors claim that we have entered a “second machine age” driven by artificial intelligence and big data (Brynjolfsson and McAfee ,2014 : Ferrari and Muffatto ,2021). In this study we rely on ML classification techniques to predict the success probability of ICOs. These models apply well to highly unstructured problems that require some form of pattern recognition and may involve missing and corrupt data (Das et al.,2018). Our models take an investors' perspective to define “Y” for successful ICOs and “N” for non-successful ICOs. With this perspective, we evaluate the effectiveness of Machine learning Classification Techniques based on its ICO models success predictions. In order to analyse this we deploy six most popular classification techniques () such as K-Nearest Neighbours (KNN) , Naive Bayes (NB), Decision Tree(DT) ,Random Forest (RF) , Support Vector Machine (SVM) , Logistic Regression (LR) classifiers . We select these techniques as they are well-known classifiers in ML and are well suitable for the given context of ICOs success prediction . Once this machine learning classifiers based models are built , effectiveness of each model is evaluated on the basis of success factors depending on various attributes contributing to the success of each ICO establishment consisting of binary function

such as hasVideo, hasReddit, hasGithub, minInvestment along with character variables of platforms on which this ICOs are performed, brandSlogans, region of procurement following the numeric function of ratings, price of ICOs in USD , teamsize,duration of ICOs , distributing percentage and coin specification numbers . In our analysis, we calculate the prediction results on basis of precision, recall, F-measure, kappa, receiver operating characteristic (ROC) curve , accuracy and AUC values of all the classifier based models in order to provide a better model service for a company or fundraising team to reach its fundraising goal successfully through ICOs .

2.Data Exploration

Analysis here makes use of information gathered from various fundraising teams and companies in the form of a dataset with attributes of ICOs project. The dataset considered here has 2767 objects to ICO projects with their success in achieving the desired fundraising goal. In this process of analysis we depend on R Studios libraries to collect and extract the data from the source, along with that we also inherit various packages into your working directory depending on the use by specific ML classification model .

Taking a look at the dataset we observe, it has been compounded with various types of variables which need to be given a clear consideration to predict the success of forthcoming ICO projects. The fundraising goals are usually dependent on many elements spending on the value of ICOs offered. Some of such elements may be in the form of numbers, character and also factors. Defining most of the predictive variables to be in numeric form, diversifying it into binary form such as ‘0’ for not having and ‘1’ for having a Video, Reddit site, GitHub, minimum investment tend to be the main attributes in deciding the success of achieving a fundraising goal. Alongwith ratings rated from 1 to 5. Relations of such binary elements can be shown below to indicate the success rate of the projects.

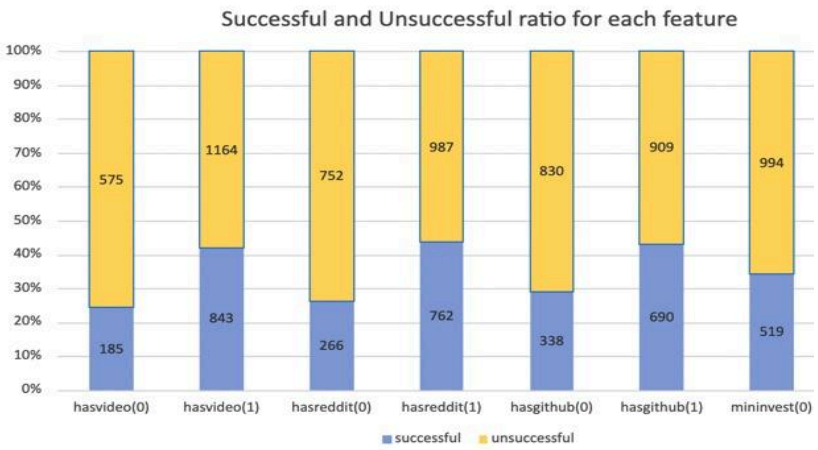

Fig 2.1

Fig. 2.2

As shown in the figure each attribute that contributes to the success of the ICO offerings is either indicated by '0' showcasing that the project does not have that specific function and '1' indicates that the attribute induces the function. For example, '1' in all the binary functional attributes indicates us that the fundraising team had a video , a Reddit site , a GitHub site and a bare minimum investment for the goal stated under the given platform on which it raises the funds . By giving a clear look at the figure , we can infer that not having any of the binary functions resulted in being unsuccessful to raise the funds for ICO offerings. More Importantly , the failure rate was high in the cases when this function deployed its attribute to contribute to the success of the project, i.e, when the function showed a value of '1'. Coming to the rating sections of the projects, we can observe that the average ratings for successful projects were

around 3.35 or more. But in the cases for unsuccessful projects it was 2.96 or less.Values adhering to this had a computed situation of being either successful or unsuccessful, depending on other attributes .

Before simulating the data towards the ML classification techniques, there is a necessity to filter the data and process the data before proceeding with the models. This preprocessing helps to identify the irregular values, missing values, and outliers which need to be dealt within the preliminary stage to get accurate and clean results. Starting with the missing values , there are many ways in which we can deal with such situations such as imputation, deletion or replacement methods. For this dataset, we are depending on the imputation method to deal with the missing values. Before defining the imputation , we need to identify the missing values in the dataset. This can be done by plotting histograms and aggregations for the data, which can be seen below .

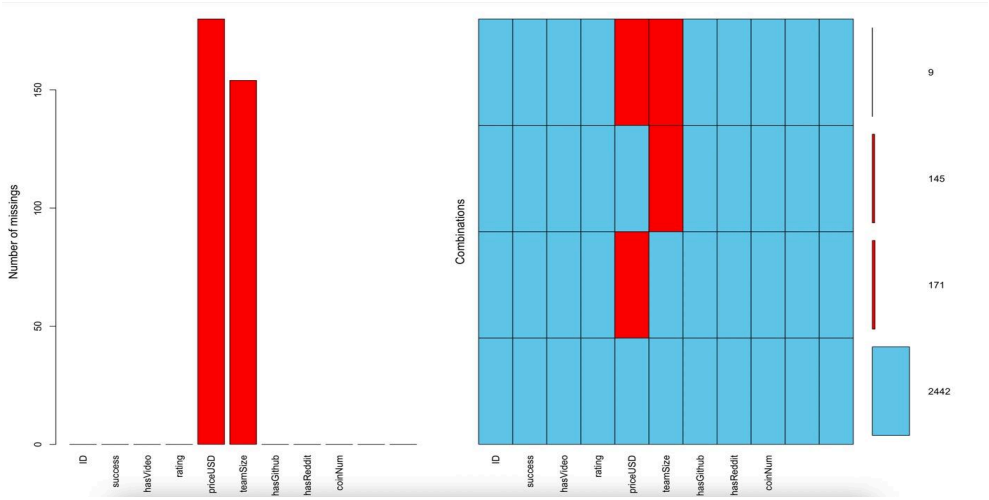

Fig 2.3

Imputation fills in one value for missing data element without developing an explicit model for partial absent data (Glas, 2010), where missing values are substituted by variable mean score of respective columns of priceUSD and teamSize. After filling out the missing data in the respective columns, we need to make one final analysation before computing the classification technique , by detecting the outliers. We can define outliers as the observations in the dataset that seem to differ significantly from another set of data (Barnett and Lewis,1994). The box plot method identifying such distant objects infers the outliers which can be clearly observed in the columns of priceUSD, teamSize, coinNum, disturbedpercentage, which can be either replaced or completely eradicated to effectively functionalize the predictive models

of the classification techniques to give more accuracy . This scattered and fabricated data is fetched to the ML classification techniques, which are explained in the next section.

3.Modelling Techniques

Classification is one of the most prominent machine learning approaches that uses derived models to predict the class of new samples using the training data. According to (Han et al.,1999) data classification is a two-set process: one being the learning set where classification model is derived from the given dataset known as Training set and second one is a classification set where the model is used to predict the class labels for a separate undisclosed dataset. The dataset used to test the ability of classification for this learned set is known as the testing set. Further, we mainly focus on effective classification techniques that will be relevant to our target area of predicting success in achieving the fundraising goals by ICOs, in the following subsections:

3.1 K-nearest neighbours (KNN)

KNN is an instance based learning classification technique that takes approximation at a local level and defers the computation until classification. It restores all the present cases in the given dataset and classifies new cases based on Euclidean distance as a similitude measure. After that, K most comparable variable called neighbours is managed by seeking the whole preparing set for another test specification point. In the end, the desired result is made by abridging the yield variable for respective K cases, dependent on a greater part for the neighbours through a majority of votes. This type of classification

technique was efficient in identifying the scams running in the ICO offering functions (Karimov and Wojchik, 2017) .

3.2 Naive Bayes (NB)

Naive Bayes classifier is a probabilistic technique ,which can forecast the class membership probabilities (Han et al. , 2011). Its ability to calculate the likelihood for each class can also define its function to handle missing attribute values by simply excluding the corresponding probabilities for those attributes. Here, the effect of one attribute on a given class is independent of those on other attributes, which can be defined as Class conditional independence, which helps to predict the class of a given dataset. Such a kind would be beneficial in predicting the success of ICOs , as (Chursook et al., 2022) used it to describe the sentimental analysis and ratings of ICOs in fundraising across various markets .

3.3 Logistic Regression (LR)

Logistic Regression being another probabilistic model, used to provide probabilities using the sigmoid function. The hypothesis of logistic regression functions between 0 and 1, where it states a relationship between categorical dependent variable and independent variable in a given dataset. Here, dependent variables are the predicting target class and independent variables are the contextual features used to predict the target class. In our context, we use all the predictors in a dataset as independent variables that are gonna function to predict the target variable of success in ICOs fundraising goal. For instance, Logistic Regression was used to over and under price the ICO offerings depending on its market downturn (Tran Bui, 2022).

3.4 Support Vector Machines (SVM)

Support Vector machines are widely used predictive classification techniques for analytics. SVM is classified as binary, i.e., storing information in terms of its class , either class 0 or class 1. For this, a hyperplane is selected in the vector machine that can indulge in variable space. Employing this hyperplane, the vector machine learning calculation finds the coefficients bringing the best remoteness of the classes. Researchers use this SVM classifier in the field of ICOs offering to decentralise the financial situation in terms of crowdfunding for start-ups(Momtaz,2022) .

3.5 Decision Tree (DT)

The most popular classification technique used for prediction named Decision Tree made by a core algorithm of ID3 stated by Quinlan (Quinlan,1986). ID3 develops a decision tree by deploying a top-down approach in which a gluttonous finding through a given training dataset is used to test each attribute at every node. Depending on ID3 an extension named C4.5 algorithm was also proposed by Quinlan(Quinlan,1993) that forms a tree from the training data in a similar manner to ID3. Following this a different algorithm named C5.0 was introduced, which is a modified version of C4.5 for predictive modelling (Kuhn et al.,2013) that is faster and more efficient compared to its descendant. Many people use different techniques for constructing the decision trees according to their analysis. In this context of fundraising goals by ICO offerings , researchers used complicated codes such as C4.5 in evaluating the model used to predict the ICO offering established for a certain period(Lahajnar and Rozanec ,2018).

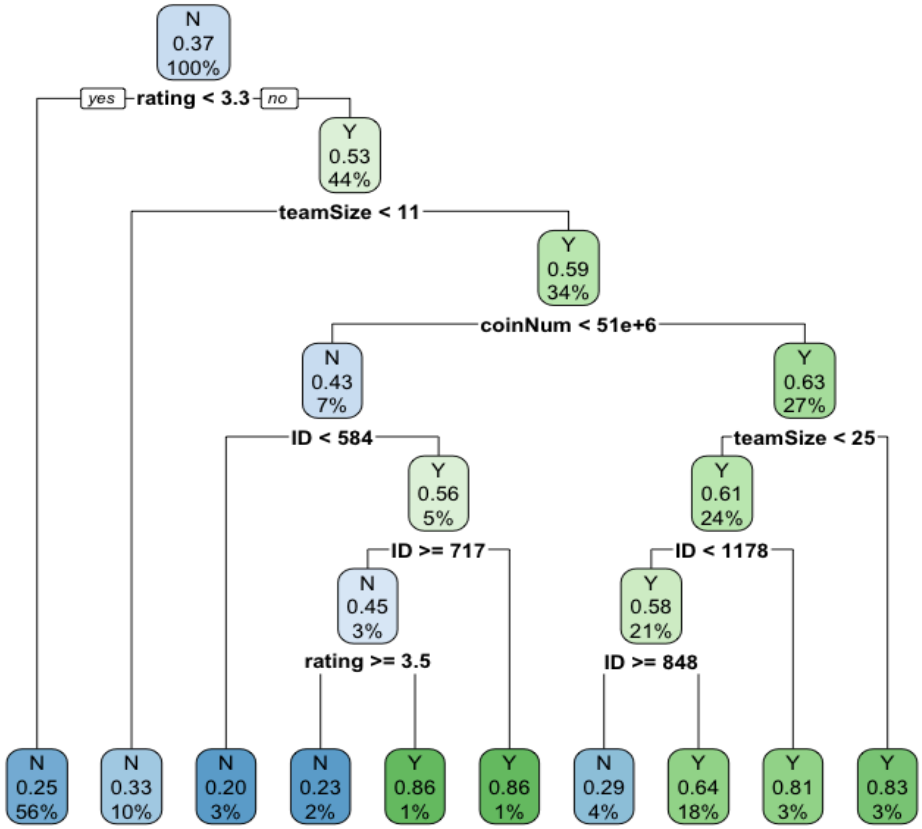

Fig 3.5.1 Decision Tree for ICO offerings

3.6 Random Forest (RF)

Random Forest was postulated by Breiman (Breiman,2001) is a troupe to ML methods that takes into account multiple learning algorithms at once to generate a prediction.Random Forest combines bootstrap aggregation(Breiman,2001)and random feature selection (Amit and Geman,1997) to form a collection of decision trees demonstrating controlled variations. It aims on generating a number of decision trees rather than forming a single decision tree, to compute the final output of success in the ICOs .Researchers use such classification techniques in predicting frauds in the ICOs, resulting in unsuccessful fundraising goals (Durr et al.,2020).

4. Evaluation Metrics

In order to quantify the accuracy of prediction by the classification techniques , we contrast the predicted out-comings with the actual out-comings and measure the accuracy of each classifier model to accurate the success of ICOs for each fundraising goal in terms of :

4.1 Precision

Precision is a count of exactness, which denotes the ratio between the number of successful ICO offerings for a given fundraising goal that are predicted correctly and the total number of ICO offerings irrespective of their success that are predicted .

$$\text{Precision} = \frac{TP}{TP+FP}$$

Where TP and FP indicate true positives and false positives, respectively.

4.2 Recall

Recall showcases the quality of completeness, which denotes the ratio between number of successful ICO offerings for a given fundraising goal that are predicted correctly and the total number of ICO offerings that are relevant to the given factor of success.

$$\text{Recall} = \frac{TP}{TP+FN}$$

Where TP and FN indicate true positives and false negatives, respectively.

4.3 ROC

Receiver operating characteristics (ROC) is used as a metric to judge the accuracy of the model depending on its AUC value, i.e, area under the curve. Effectivity of the classification model can be directly proportional to its AUC value corresponding to its accuracy.

4.4 F-measure

F-measure can be stated as a metric that merges precision and recall in a single score , which can be seen as a harmonic mean of precision and recall.

$$\text{F-measure} = 2 * \frac{Precision * Recall}{Precision + Recall}$$

4.5 Kappa

Kappa is a computational metric that contrasts an observed accuracy with an expected accuracy, which can be mathematically denoted as :

$$\text{Kappa} = \frac{(observed\ accuracy - expected\ accuracy)}{(1 - expected\ accuracy)}$$

To show the effectiveness of the individual machine learning classifier model to predict the success of ICO offering for a given fundraising goal , can be shown in the following table which depicts the results of Classification models for their accuracy level at which a given prediction is made . For each model, we use the same datasets in order to make a fair comparison within the classification techniques. For

evaluation, we deploy the tenfold cross validation technique on each ICO dataset considered ,in which the data is split into a log of 10 sets of size N/10. Further which it trains each model on 7 sets and tests it using the remaining three sets. To show the effectiveness of each classification model, we calculate the prediction outcomes in terms of precision, recall, F-measure, Kappa, receiver operating characteristic (ROC) curve and area under the curve value for the model.

Classifier	CCI (%)	ICI (%)	AUC value(%)
KNN	100	0	100
NB	63	37	70.5
DT	81	19	61
RF	69	31	65
SVM	71	29	70
LR	66	35	67

Table 4.1

Table shows prediction results for each model in terms of CCI (correctly classified instances) rate, ICI (incorrectly classified instances) rate and area under the curve for obtaining ROC curve for respective ICO offering sets to complete the goal established. From the table, we can clearly observe that KNN completely overshadows other models in terms of prediction with an accuracy of 100%, following lowest ICI rate and large amount of area following under the ROC curve. Decision tree imposes the second best

accuracy rate, whereas Naive-Bayes showcases poor performance compared to other classification models .

5. Effective comparison

Model effectiveness of each ML classifier for individual fundraising goals to raise ICOs can be shown by the above figure, where a relative comparison of precision, recall, F - measure and kappa for various classification models are observed. For each prediction model, we use the same amount of training and testing dataset for fair evaluation among the models. For individual comparison, we also depict the relative comparison of average results for precision, recall, F-measure and kappa for all the datasets observed .

Metrics	**NB**	**DT**	**RF**	**KNN**	**SVM**	**LR**
Precision	0.98	0.86	0.825	1	0.72	0.67
Recall	0.65	0.832	0.736	1	0.90	0.84
F-measure	0.77	0.845	0.78	1	0.80	0.75
Kappa	0.12	0.60	0.26	1	0.30	0.32

Table 5.1

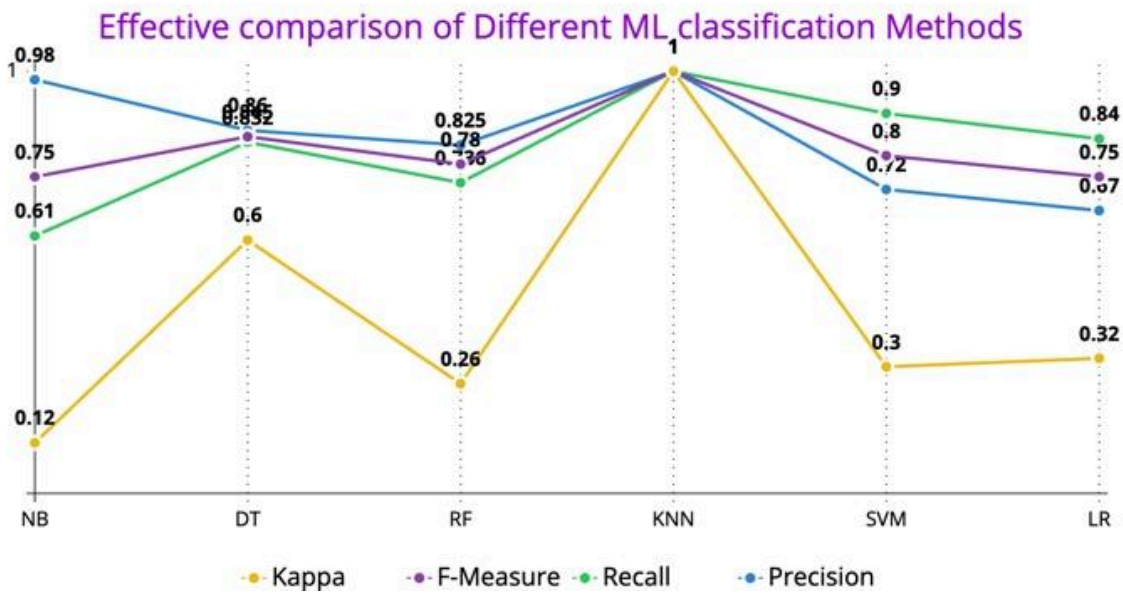
Effective comparison of Different ML classification Methods
1
0.98
0.75
0.61
0.12
0.6
0.825
0.78
0.26
1
0.9
0.8
0.72
0.3
0.84
0.75
0.67
0.32
NB
DT
RF
KNN
SVM
LR
Kappa
F-Measure
Recall
Precision

Fig. 5.1

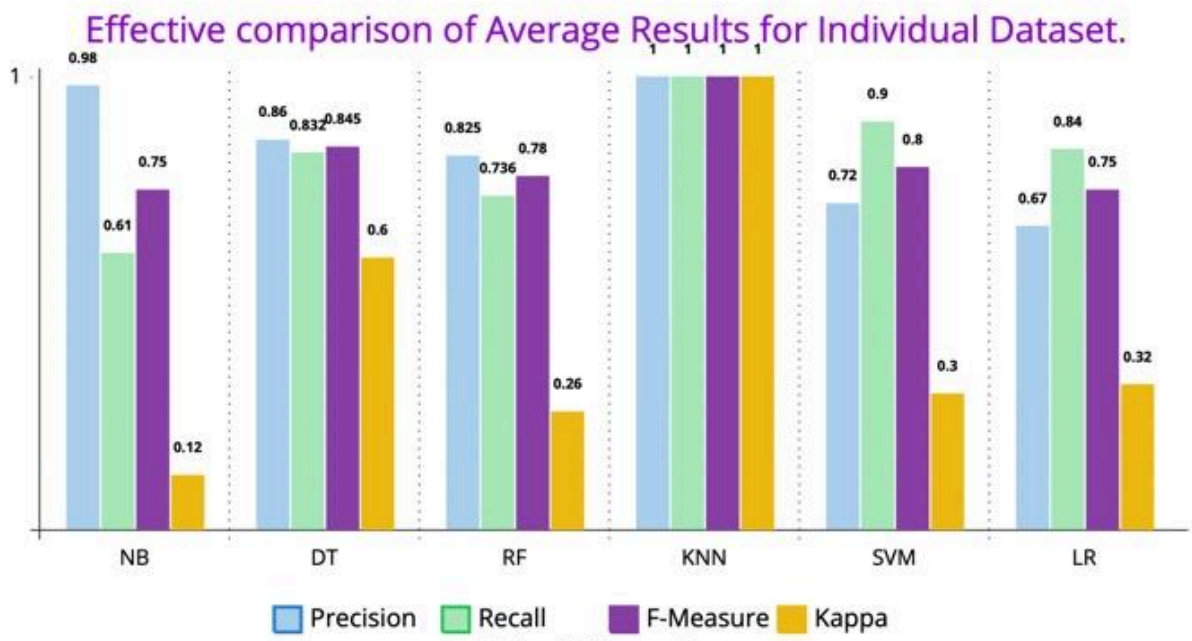
Effective comparison of Average Results for Individual Dataset.
1
0.98
0.61
0.75
0.12
0.86
0.832
0.845
0.6
0.825
0.736
0.78
0.26
1
1
1
1
0.72
0.9
0.8
0.3
0.67
0.84
0.75
0.32
NB
DT
RF
KNN
SVM
LR
Precision
Recall
F-Measure
Kappa

Fig 5.2

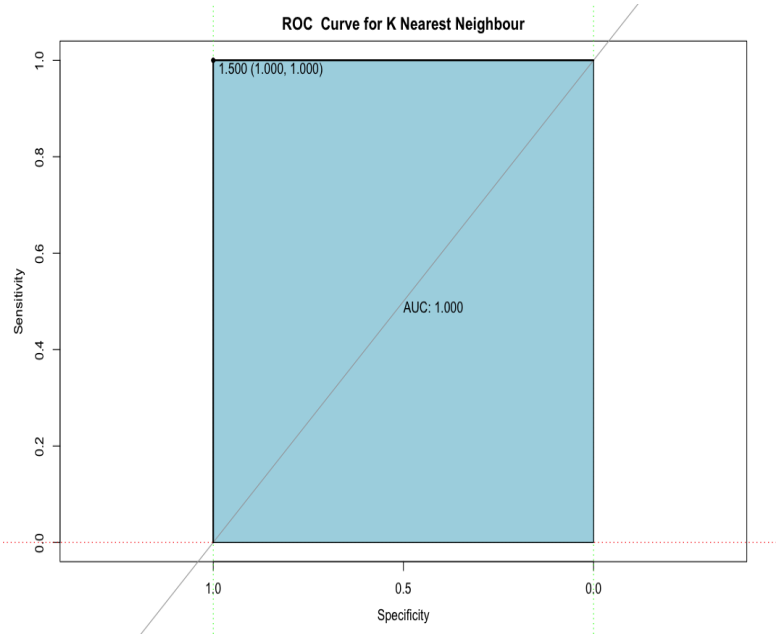
ROC Curve for K Nearest Neighbour
1.500 (1.000, 1.000)
AUC: 1.000
Sensitivity
Specificity
1.0
0.8
0.6
0.4
0.2
0.0
1.0
0.5
0.0

Fig 5.3

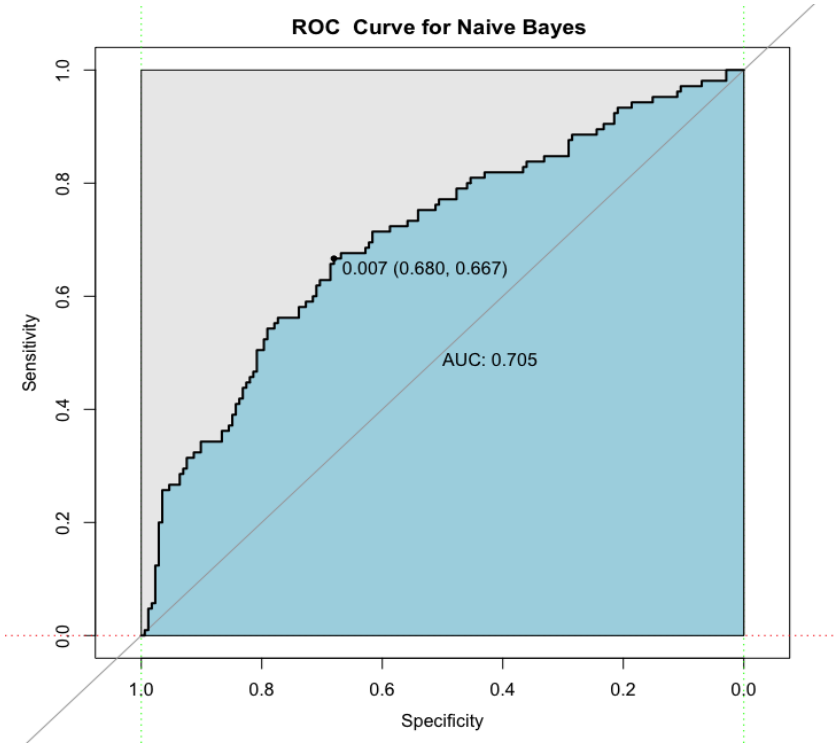
ROC Curve for Naive Bayes
0.007 (0.680, 0.667)
AUC: 0.705
Sensitivity
Specificity
1.0
0.8
0.6
0.4
0.2
0.0
1.0
0.8
0.6
0.4
0.2
0.0

Fig 5.4

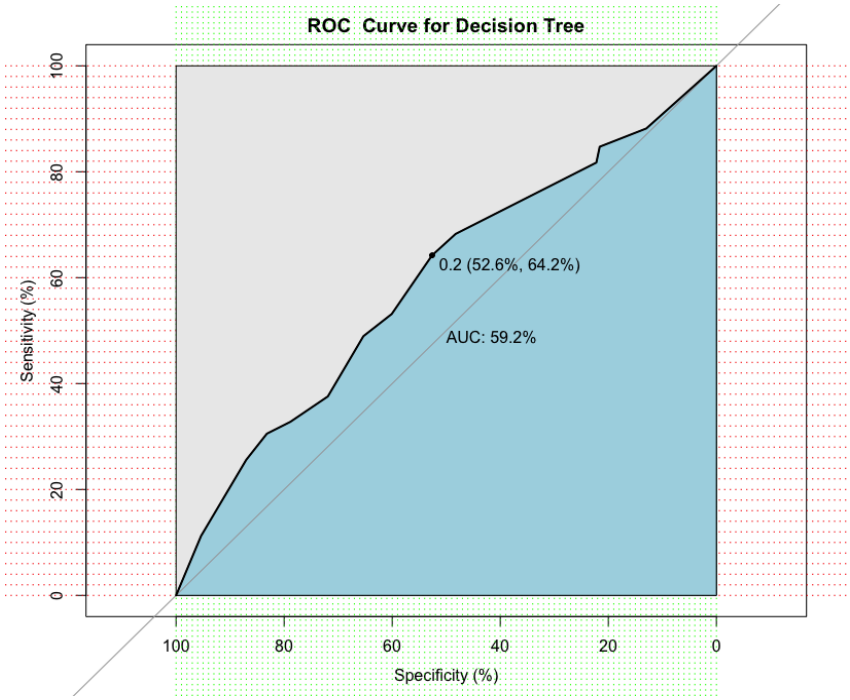
ROC Curve for Decision Tree
0.2 (52.6%, 64.2%)
AUC: 59.2%
Sensitivity (%)
Specificity (%)
100
80
60
40
20
0

Fig 5.5

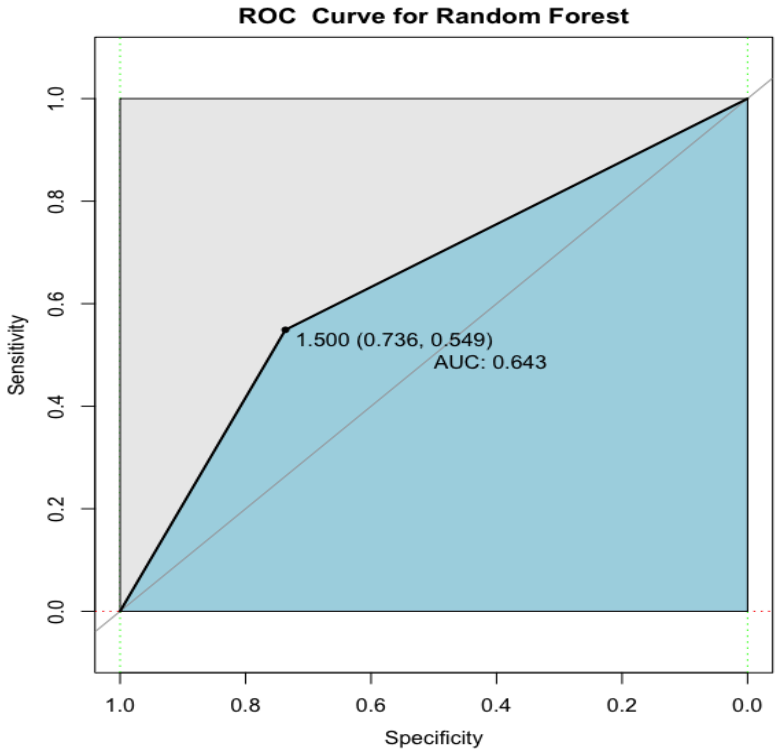
ROC Curve for Random Forest
1.500 (0.736, 0.549)
AUC: 0.643
Sensitivity
Specificity
1.0
0.8
0.6
0.4
0.2
0.0

Fig 5.6

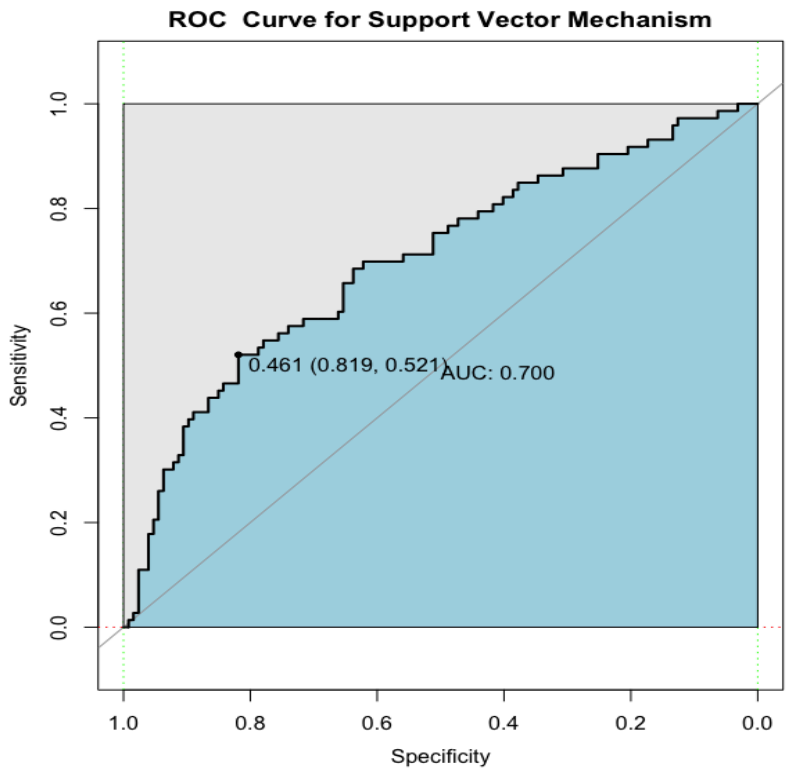
ROC Curve for Support Vector Mechanism
Sensitivity
Specificity
0.461 (0.819, 0.521)
AUC: 0.700
1.0
0.8
0.6
0.4
0.2
0.0

Fig 5.7

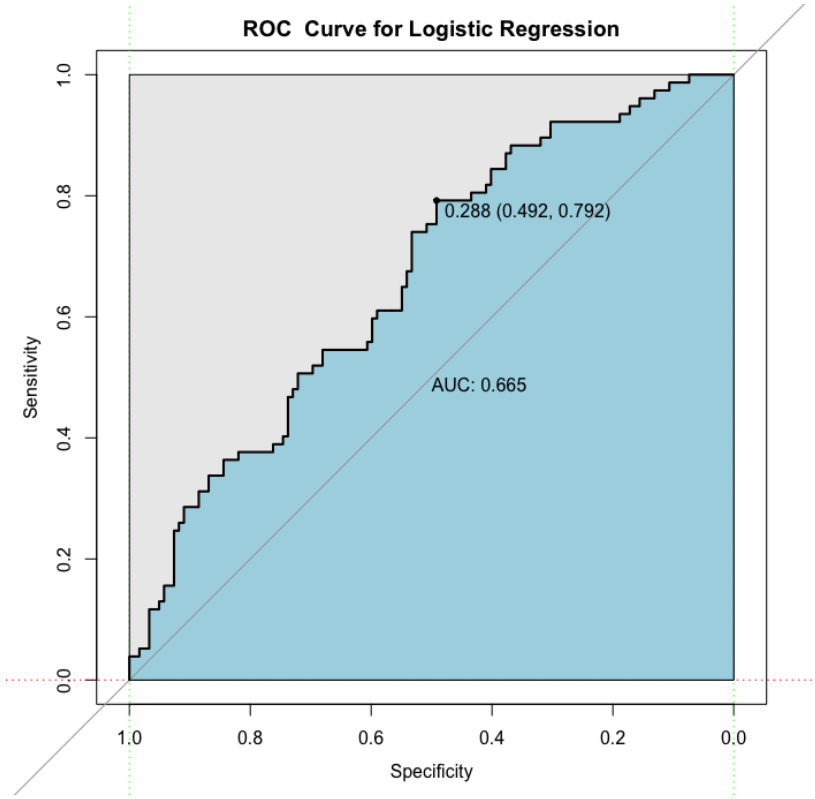
ROC Curve for Logistic Regression
Sensitivity
Specificity
0.288 (0.492, 0.792)
AUC: 0.665
1.0
0.8
0.6
0.4
0.2
0.0

Fig 5.8

As seen in the ROC curves of all the six computed classification techniques, it is clearly adequate that KNN with highest precision of sensitivity to specificity overcasts all the other models. One of the main reason for this behaviour of curves is its defining predictive models which introduces ROC function with consideration of all the present attributes in the dataset against the tested predictive model of knowing success of the companies in completing the fundraising goal with ICO offerings with the best AUC(area under the curve) value .

From all the figures, we can draw an outline that KNN out shadows all the other classification techniques in predicting the success of ICO offerings for given fundraising goals with the highest precision , recall, kappa, and F-measure values .

5.1 KNN vs Baseline Classifiers

As seen in the figures , the accuracy along with the cumulation of AUC value ,precision, recall, f-measure, and kappa values of Naive Bayes, Random Forest and Logistic Regression show significantly low values when compared to the other stated models, making them the baseline classifiers with low context accuracy. The main reason for this is that all the three classification models define their predictive model only in terms of ICO success without involving other attributes such as priceUSD and teamSize to define the fundraising goals which drops down its accuracy from 63% to 70%, as it can be clearly seen in KNN where the training model introduces all variables regardless of their strings to compute the success of fundraising goals, thus simulating in high accuracy.

5.2 KNN vs Highline Classifiers

Apart from baseline classifiers discussed above, classification models such as Decision Tree and Support Vector Mechanism give good significant predictive results ranging the accuracy from 70-85%, proclaiming themselves as the highline classifiers. As seen from the figures above,

these models give good personalised results for the success of ICO offerings, but in comparison with the KNN model they lag behind in building a highly accurate predictive model. Hence, the KNN model best suits the context of predicting the success of fundraising teams on ICO offerings, depending on their fundraising goals .

6. Conclusion

Overall, the study influences the importance of Machine learning classification techniques in predicting the success of the ICO offerings for given fundraising goals. All the classification techniques possess prominent accuracy and behaviour towards predicting the success of ICO offerings. The metrics used in the study successfully help us in evaluating all the models by various attributes, making the specifications clear to all the fundraising teams in future cases. ROC curves bounding over 60% demonstrate the ability of all the models being efficient in the context of displaying fraudulent and non-fraudulent projects. Amongst all, KNN displays the maximum accuracy of 100% in outlining the predictive behaviour. The fundraising teams and the companies for future work should be careful in evaluating their success for the goals with the help of models by deliberation of the false negative and false positive values, resulting in misconducting errors in the prediction models. A proper pre-processing of data should be procured while engaging with the models by detaching the outliers, or else it would reflect in reducing the accuracy of the models and later produce a false result for the ICO offerings compounding to irrationality in the teams. In the given context, all the six models analysed as Ultimate Classifier (KNN), Highline Classifier (DT, SVM) and Baseline Classifier (NB, RF, LR)

help the fundraising teams and companies in determining their success for ICOs offered according to the fundraising goals .

7. References

- Chohan, U.W., 2017. *Initial coin offerings (ICOs): Risks, regulation*. and accountability, Discussion Paper Series: Notes on the 21st Century.
- Momtaz, P.P., 2021. Entrepreneurial finance and moral hazard: evidence from token offerings. *Journal of Business Venturing*, *36*(5), p.106001.
- Ferrati, F. and Muffatto, M., 2021. Entrepreneurial finance: emerging approaches using machine learning and big data. *Foundations and Trends® in Entrepreneurship*, *17*(3), pp.232-329.
- Agrawal, R., 2020. Fundamentals of machine learning. In *Machine Learning for Healthcare* (pp. 1-16). Chapman and Hall/CRC.
- Brynjolfsson, E. and McAfee, A., 2014. *The second machine age: Work, progress, and prosperity in a time of brilliant technologies*. WW Norton & Company.
- Das, S., Datta, S. and Chaudhuri, B.B., 2018. Handling data irregularities in classification: Foundations, trends, and future challenges. *Pattern Recognition*, *81*, pp.674-693.
- Glas, C.A. and van der Linden, W.J., 2010. Marginal likelihood inference for a model for item responses and response times. *British Journal of Mathematical and Statistical Psychology*, *63*(3), pp.603-626.

- Barnett, V. and Lewis, T., 1994. *Outliers in statistical data* (Vol. 3, No. 1). New York: Wiley.
- Han, J., Pei, J. and Kamber, M., 1999. Data mining: concepts and techniques. 2011.
- Karimov, B. and Wójcik, P., 2021. Identification of scams in Initial Coin Offerings with machine learning. *Frontiers in Artificial Intelligence*, p.120.
- Chursook, A., Dawod, A.Y., Chanaim, S., Naktnasukanjn, N. and Chakpitak, N., 2022. Twitter Sentiment Analysis and Expert Ratings of Initial Coin Offering Fundraising: Evidence from Australia and Singapore Markets. *TEM Journal*, *11*(1), p.44.
- Tran Bui, M.A., 2022. *Initial Exchange Offerings and the Presence of Underpricing* (Doctoral dissertation, Hochschule Rhein-Waal).
- Momtaz, P.P., 2022. How Efficient is Decentralized Finance (DeFi)?. *Available at SSRN 4063670*.
- Quinlan, J.R., 1986. Induction of decision trees. *Machine learning*, *1*, pp.81-106.
- Kuhn, M., Johnson, K., Kuhn, M. and Johnson, K., 2013. Classification trees and rule-based models. *Applied predictive modeling*, pp.369-413.
- Lahajnar, S. and Rožanec, A., 2018. Initial coin offering (ICO) evaluation model. *Investment Management & Financial Innovations*, *15*(4), p.169.
- Breiman, L., 2001. Random forests. *Machine learning*, *45*, pp.5-32.
- Amit, Y. and Geman, D., 1997. Shape quantization and recognition with randomized trees. *Neural computation*, *9*(7), pp.1545-1588.
- Dürr, A., Griebel, M., Welsch, G. and Thiesse, F., 2020. Predicting Fraudulent initial Coin Offerings using Information Extracted from Whitepapers. In *ECIS*.

8.Appendix

Pre-Processing of Data

```r
icobench<- read.csv("Downloads/Machinelearningcourseworkright.csv")
str(icobench)
sum(complete.cases(icobench))
sum(!complete.cases(icobench))
summary(icobench)
View(icobench)
library("VIM")
aggr(icobench,numbers=TRUE,prop=FALSE)
qicobench<-icobench[which(icobench$hasVideo==1&icobench$rating>=4.0),]
hist(icobench$priceUSD)
si_icobench<-icobench
si_icobench$priceUSD[is.na(si_icobench$priceUSD)]<-mean(si_icobench$priceUSD,na.rm = TRUE)
hist(icobench$teamSize)
si_icobench$teamSize[is.na(si_icobench$teamSize)]<-mean(si_icobench$teamSize,na.rm = TRUE)
sum(complete.cases(si_icobench))
data<-si_icobench
boxplot(data$priceUSD)
boxplot(data$priceUSD,plot = FALSE)$out # priceUSD,teamsize,coinnum,dirtubutedpercentage are the outliers
outlier<-boxplot(data$priceUSD,plot = FALSE)$out
x<-data
x<-x[-which(x$priceUSD %in% outlier),]
length(x$priceUSD)
length(x$teamSize)
boxplot(x$priceUSD)
sum(!complete.cases(x))
outliers<-boxplot(x$teamSize,plot = FALSE)$out
y<-x
y<-y[-which(y$teamSize %in% outliers),]
length(y)
length(y$priceUSD)
out<-boxplot(y$coinNum,plot = FALSE)$out
z<-y
z<-z[-which(z$coinNum %in% out),]
length(z$teamSize)
outl<-boxplot(z$distributedPercentage,plot = FALSE)$out
w<-z
w<-w[-which(w$distributedPercentage %in% outl),]
sum(!complete.cases(w))
w<-data
```

Fig. 8.1 Code for Pre- Processing of the data

KNN Classifier

```r
library(class)
library(pROC)
library(caret)
library(knitr)
library(ggplot2)
library(tidyr)
library(dplyr)
library(ROCR)
library(stringi)
library(e1071)
kable(table(w$success),col.names = c("Success","priceUSD",align='1'))
ggplot(gather(w[,2:ncol(data)]), aes(value)) +
  geom_histogram(bins = 5, fill = "blue", alpha = 0.6) +
  facet_wrap(~key, scales = 'free_x')
w$success=as.factor(w$success)
levels(w$success)=make.names(levels(factor(w$success)))
set.seed(101)
Index<- sample(1:nrow(w),size = round(0.9*nrow(w)),replace=FALSE)
train=w[Index,]
test<-w[-Index,]
rm(Index)
K=47
model<-knn(train = train[,-ncol(train)],test =test[,-ncol(test)],cl=train[,ncol(train)],k=K)
levels(train$success)=make.names(levels(factor(train$success)))
levels(test$success)<-make.names(levels(factor(test$success)))
repeats=3
numbers=10
tunel=10
set.seed(123)
x<-trainControl(method = "repeatedcv",
                number =numbers,
                repeats = repeats,
                classProbs = TRUE,
                summaryFunction = twoClassSummary)
knn<-train(success~.,data=train,method="knn",preProcess=c("center","scale"),
          trControl=x,
           metric='ROC',
           tuneLength=tunel)
plot(knn)
```

Fig. 8.2 Codes for KNN

```r
plot(knn)
test_pred<-predict(knn,test)
test_pred
confusionMatrix(test_pred,test$success,positive='Y')
library(pROC)
test_pred<-as.numeric(test_pred)
test$success<-as.numeric(test$success)
roc<-roc(test_pred,test$success)
plot(roc,print.auc = TRUE,auc.polygon = TRUE,grid = c(1,2),grid.col = c("green","red"),max.auc.polygon = TRUE,auc.polygon.col = "lightblue"
auc(roc)
```

Fig. 8.3 Codes for KNN

```
> confusionMatrix(test_pred,test$success,positive='Y')
Confusion Matrix and Statistics

          Reference
Prediction   N   Y
         N 163   0
         Y   0 114

               Accuracy : 1
                 95% CI : (0.9868, 1)
    No Information Rate : 0.5884
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 1

 Mcnemar's Test P-Value : NA

            Sensitivity : 1.0000
            Specificity : 1.0000
         Pos Pred Value : 1.0000
         Neg Pred Value : 1.0000
             Prevalence : 0.4116
         Detection Rate : 0.4116
   Detection Prevalence : 0.4116
      Balanced Accuracy : 1.0000

       'Positive' Class : Y
```

Fig 8.4 Confusion Matrix for KNN

Naive Bayes Classifier

```r
#Naive bayes
library(tm)
library(caret)
library(e1071)
library(pROC)
length(icobench$platform)
set.seed(123)
train_idx<-sample(nrow(w),0.85*nrow(w))
train_data<-w[train_idx,]
test_data<-w[-train_idx,]
model<-naiveBayes(success~.,data = train_data)
pred<-predict(model,newdata = test_data,type = "class")
pred<-as.factor(pred)
test_data$success<-as.factor(test_data$success)
confusionMatrix(test_data$success,pred,positive='Y')
kappa(pred,test_data$success)
prediction<-predict(model,newdata = test_data,type = "raw")
predict<-prediction(pred,test_data$success)
confusionMatrix(pred,test_data$success,positive="Y")
perf<-performance(pred,measure = "tpr",x.measure = "fpr")
prediction=prediction[,2]
library(caret)
prediction<-predict(model,newdata= test_data,type="raw")
roc_object<-roc(test_data$success,prediction[,2])
plot.roc(roc_object,print.auc = TRUE,auc.polygon = TRUE,grid= c(1,2),grid.col = c("green","red"),max.auc.polygon=TRUE,auc.polygon.col = "li
plot.roc(rl,print.auc = TRUE,auc.polygon = TRUE,grid = c(1,2),grid.col = c("green","red"),max.auc.polygon = TRUE,auc.polygon.col = "lightbl
plot(roc_object)
auc(roc_object)
```

Fig 8.5 Codes for NB

```
> confusionMatrix(test_data$success,pred,positive='Y')
Confusion Matrix and Statistics

          Reference
Prediction   N   Y
         N 170   2
         Y 101   4

               Accuracy : 0.6282
                 95% CI : (0.5683, 0.6852)
    No Information Rate : 0.9783
    P-Value [Acc > NIR] : 1

                  Kappa : 0.0324

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.66667
            Specificity : 0.62731
         Pos Pred Value : 0.03810
         Neg Pred Value : 0.98837
             Prevalence : 0.02166
         Detection Rate : 0.01444
   Detection Prevalence : 0.37906
      Balanced Accuracy : 0.64699

       'Positive' Class : Y
```

Fig 8.6 Confusion Matrix for NB

Decision Tree

```
#Decision tree
set.seed(123)
trainindex<-createDataPartition(w$success,p=0.3,list = FALSE)
train<-w[trainindex,]
test<-w[-trainindex,]
dt_model<-rpart(success~.,data=train,method="class")
dt.preds<-predict(dt_model,test,type = "prob")
library(rpart.plot)
rpart.plot(dt_model)
library(C50)
library(tidyverse)
library(party)
library(mlbench)
library(tree)
printcp(dt_model)
p<-predict(dt_model,train,type='class')
confusionMatrix(p,train$success,positive = 'Y')
pl<-dt.preds[,2]
r<-multiclass.roc(test$success,pl,percent=TRUE)
roc<-r[['rocs']]
rl<-roc[[1]]
par(mar = c(5.1, 4.4, 4.1, 1.9))
plot.roc(rl,print.auc = TRUE,auc.polygon = TRUE,grid = c(1,2),grid.col = c("green","red"),max.auc.polygon = TRUE,auc.polygon.col = "lightbl
auc(r)
```

Fig 8.7 Codes for DT

```
> confusionMatrix(p,train$success,positive = 'Y')
Confusion Matrix and Statistics

          Reference
Prediction   N   Y
         N 104  17
         Y  21  58

               Accuracy : 0.81
                 95% CI : (0.7487, 0.8619)
    No Information Rate : 0.625
    P-Value [Acc > NIR] : 1.103e-08

                  Kappa : 0.5989

 Mcnemar's Test P-Value : 0.6265

            Sensitivity : 0.7733
            Specificity : 0.8320
         Pos Pred Value : 0.7342
         Neg Pred Value : 0.8595
             Prevalence : 0.3750
         Detection Rate : 0.2900
   Detection Prevalence : 0.3950
      Balanced Accuracy : 0.8027

       'Positive' Class : Y

>
```

Fig 8.8 Confusion matrix for DT

Random Forest

```r
#Random Forest Classifers
set.seed(1234)
trainindex<-sample(1:nrow(w),round(0.8*nrow(w)),replace = FALSE)
trainData<-w[trainindex,]
testData<-w[-trainindex,]
library(randomForest)
library(pROC)
unique(testData$success)
rf<-randomForest(success~ .,data = trainData,ntree=500)
rfPred<-predict(rf,testData,type = "class")
rfPred<-as.factor(rfPred)
testData$success<-as.factor(testData$success)
confusionMatrix(rfPred,testData$success,positive = "Y")
rfPred<-as.numeric(rfPred)
testData$success<-as.numeric(testData$success)
roc <- roc (rfPred,testData$success)
plot.roc(roc,print.auc = TRUE,auc.polygon = TRUE,grid = c(1,2),grid.col = c("green","red"),max.auc.polygon = TRUE,auc.polygon.col = "lightb
auc(roc)
```

Fig 8.9 Codes for RF

```
> confusionMatrix(rfPred,testData$success,positive = "Y")
Confusion Matrix and Statistics

          Reference
Prediction   N   Y
         N 109  39
         Y  23  28

               Accuracy : 0.6884
                 95% CI : (0.6191, 0.7521)
    No Information Rate : 0.6633
    P-Value [Acc > NIR] : 0.25130

                  Kappa : 0.2589

 Mcnemar's Test P-Value : 0.05678

            Sensitivity : 0.4179
            Specificity : 0.8258
         Pos Pred Value : 0.5490
         Neg Pred Value : 0.7365
             Prevalence : 0.3367
         Detection Rate : 0.1407
   Detection Prevalence : 0.2563
      Balanced Accuracy : 0.6218

       'Positive' Class : Y
```

Fig 8.10 Confusion Matrix RF

Support Vector Mechanism

```r
#Suppourt Vector Mechanism
library(e1071)
library(pROC)
set.seed(123)
trainindex<-sample(nrow(w),0.8*nrow(w))
train_data<-w[trainindex,]
test_data<-w[-trainindex,]
svm_model<-svm(success~.,data = train_data,kernel="linear",cost=10,probability=TRUE)
svm_predictions<-predict(svm_model,test_data)
svm_probabilities<-attr(predict(svm_model,test_data,probability=TRUE),"probabilities")
pred<-prediction(svm_probabilities[,2],test_data$success)
roc_obj<-roc(test_data$success,svm_probabilities[,2])
auc_value<-auc(roc_obj)
plot(roc_obj,print.auc = TRUE,auc.polygon = TRUE,grid = c(1,2),grid.col = c("green","red"),max.auc.polygon = TRUE,auc.polygon.col = "lightb
auc_value
```

Fig 8.11 Codes for SVM

```
> confusionMatrix(svm_predictions,test_data$success,positive="Y")
Confusion Matrix and Statistics

          Reference
Prediction   N   Y
         N 115  46
         Y  12  27

               Accuracy : 0.71
                 95% CI : (0.6418, 0.7718)
    No Information Rate : 0.635
    P-Value [Acc > NIR] : 0.01548

                  Kappa : 0.3056

 Mcnemar's Test P-Value : 1.47e-05

            Sensitivity : 0.3699
            Specificity : 0.9055
         Pos Pred Value : 0.6923
         Neg Pred Value : 0.7143
             Prevalence : 0.3650
         Detection Rate : 0.1350
   Detection Prevalence : 0.1950
      Balanced Accuracy : 0.6377

       'Positive' Class : Y
```

Fig 8.12 Confusion Matrix for SVM

Logistic Regression

```
#Logistic Regression
install.packages("neuralnet")
library(neuralnet)
library(pROC)
set.seed(123)
trainIndex<-sample(1:nrow(w),round(0.7*nrow(w)))
train<-w[trainIndex,]
test<-w[-trainIndex,]
formula<-as.formula("success ~ hasVideo + rating +priceUSD+ teamSize+hasGithub+hasReddit+coinNum+minInvestment+distributedPercentage")
model<-glm(formula,data = train,family = binomial(link = "logit"))
logprediction<-predict(model,test,type = "response")
logpredictionbinary<-ifelse(logprediction>0.5,1,0)
confusionMatrix<-table(logpredictionbinary,test$success)
logpredictionbinary<-as.factor(logpredictionbinary)
test$success<-as.factor(test$success)
confusionMatrix
(logpredictionbinary,test$success,positive="Y")
259/(259+)
observedAgreement <- sum(diag(confusionMatrix)) / sum(confusionMatrix)
chanceAgreement <- sum(rowSums(confusionMatrix) * colSums(confusionMatrix)) / sum(confusionMatrix)^2
kappaValue <- (observedAgreement - chanceAgreement) / (1 - chanceAgreement)
kappaValue
rocCurve<-roc(test$success,logprediction)
plot.roc(rocCurve,print.auc = TRUE,auc.polygon = TRUE,grid = c(1,2),grid.col = c("green","red"),max.auc.polygon = TRUE,auc.polygon.col = "l
aucVal<-auc(rocCurve)
aucVal
```

Fig 8.13 Codes for LR

```
> confusionMatrix

logpredictionbinary   N   Y
                  0 321 121
                  1  65  91
```

Fig 8.14 Confusion Matrix for LR